An Auditable Clinical Data Warehouse and Prospective Study Workbench for MIMIC-IV-Shaped Data

Pronab Chandra Roy*

Technical research manuscript — 18 October 2026

Abstract

Clinical prediction studies can fail before a model is fitted: source tables may be incomplete, the cohort can be defined at the wrong grain, measurements may become available after the intended prediction time, and a polished performance summary can conceal these choices. This manuscript describes a local, open-source clinical data warehouse and a narrowly scoped research workbench designed to make those choices inspectable. The system loads MIMIC-IV-shaped files into Bronze, Silver, and Gold layers in DuckDB; offers aggregate analytics through a command-line interface, FastAPI, and Streamlit; and implements a synthetic first-ICU-stay mortality experiment with a versioned time contract. A deterministic invented-data generator permits end-to-end execution without access to the credentialed MIMIC-IV database. The prospective study excludes events outside the first 24 hours or unavailable by the prediction time, separates a patient-level train/validation/test partition, and exports an allowlisted aggregate package. A sample synthetic run has 560 training, 120 validation, and 120 test rows; its test area under the precision–recall curve is 0.109. That figure describes recovery of invented patterns, not clinical accuracy. The central contribution is an executable pathway from source provenance to cohort selection, feature timing, evaluation, and reviewable evidence. The manuscript also identifies remaining threats to validity, including incomplete data fingerprints, a narrow task registry, simulated rather than independent hospital sites, and the absence of any credentialed-data validation.

Keywords: clinical data warehouse; electronic health records; MIMIC-IV; temporal leakage; synthetic data; reproducible research; data quality.

1 Research motivation and scope

Electronic health records (EHRs) are attractive for retrospective and predictive research, but their structure reflects care delivery and operational recording rather than a ready-made study design. MIMIC-IV is a well-described EHR resource containing hospital and intensive-care data [1, 2]. Access to the full release requires PhysioNet credentialing, training, and a data use agreement [2]. These access conditions have a direct methodological consequence for this project: no result reported here is a result on the credentialed MIMIC-IV patient population. The repository's tiny synthetic demonstration and its larger invented research generator are engineering fixtures. The separately available MIMIC-IV Clinical Database Demo is useful for learning the source layout, but it is not the full cohort [3].

The project's research question is therefore deliberately about *research infrastructure*: can a local warehouse make the route from source tables to a time-bounded ICU prediction study explicit, reproducible, and inspectable, and can controlled synthetic faults show where that route breaks? The implemented flagship experiment asks how first-24-hour measurement availability and data quality affect a synthetic ICU mortality study. This differs from claiming that the resulting classifier is a clinically useful mortality model. A reviewer should be able to locate the study definition,

*Project repository: https://github.com/pronabroy360/MIMIC-IV-Clinical-DW.

explain why each patient is included, identify the exact event and availability windows for each predictor, and inspect the evaluation and release boundary.

The contribution has three parts. First, a layered warehouse normalizes source-shaped records and supports aggregate clinical analytics. Second, a fixed prospective study contract ties cohort grain, prediction time, permitted features, outcome, and split unit to executable code. Third, synthetic scenarios and aggregate artifacts make data-quality and distribution-shift questions discussable without exporting patient-level records. These parts share one local configuration and service layer across command-line, API, and dashboard entry points. The implementation is a research prototype, not a deployed clinical decision-support system.

2 System architecture and data provenance

2.1 Execution modes and layered warehouse

The repository supports a small invented demo seed, an isolated 1,000-patient invented research generator, ingestion of locally available source files, and a LocalStack S3 path. The locally available full-file path is useful only to a credentialed operator who independently supplies permitted files; it is not evidence that those files exist in this repository. The BigQuery component is a metadata scaffold rather than a remote clinical-data execution path. The research generator writes source-shaped CSV files to an ignored raw-data directory and builds an isolated DuckDB warehouse, avoiding accidental mixing with the standard six-patient demo.

Bronze tables preserve source-like rows and add ingestion metadata including source file, load time, module, and pipeline run identifier. Missing optional inputs produce empty placeholders, which lets SQL build but must not be interpreted as observed zero clinical events. Silver tables standardize patient, admission, ICU-stay, laboratory, chart, and intervention entities and retain the timestamps needed downstream. Gold contains descriptive marts such as patient, admission, ICU-stay, and first-24-hour summaries. Research feature construction is a separate path: descriptive Gold summaries may describe the whole stay, whereas the prospective table must obey its own cutoff. SQL and Python modules implement the transformations; the `run-all` command orchestrates ingestion, Gold construction, validation, and reporting.

Stage	Principal responsibility	Review evidence
Source/Bronze	Discover and ingest MIMIC-shaped files; record provenance	Source coverage and ingestion report
Silver	Type, join, and normalize clinical entities and event times	Core keys, timestamps, and joins
Gold	Build descriptive aggregate marts	Row counts and aggregate dashboard/API views
Research	Compile cohort; enforce feature timing; fit a fixed task model	Study definition, cohort flow, timing and feature audits
Publication	Copy only named aggregate artifacts	Manifest, quality summary, SHA-256 checksums

Table 1: The local evidence chain. Empty placeholders preserve buildability, but coverage still needs explicit inspection.

Configuration files select database and output paths; the Makefile exposes the operator workflow. The dashboard presents aggregate views of overview, admissions, ICU, laboratories and vitals, cohorts, and the research workbench. FastAPI exposes health, analytics, cohort, and study routes. Their shared research service validates definitions and runs the same cohort and feature code. A rule-based natural-language query assistant supports predefined aggregate questions. SQL guardrails reject destructive statements, unbounded raw-row patterns such as `SELECT *`, Bronze access, raw identifier projection, and nonaggregate queries. These are useful application boundaries, although they do not constitute a formal privacy guarantee or replace access control on a real deployment.

2.2 Two distinct synthetic workflows

The earlier teaching workshop generates 1,000 invented patients and source-like admissions, diagnoses, procedures, services, labs, charts, and inputs. Its index unit is the first hospital admission per patient. It studies hospital length of stay, in-hospital death, urgent readmission within 30 days, and a compact synthetic diagnosis class. The readmission label is null for an index death and requires a later urgent admission strictly after discharge and within 30 days. The workshop includes a 73-candidate feature catalog, descriptive plots, baseline models, a fixed patient-level split, and repeated cross-validation. Some workshop predictors have retrospective or incomplete prospective availability; the compact diagnosis class is a teaching target, not a validated phenotype. Consequently this path should be read as a tutorial and regression fixture, not as the prospective task described next.

The flagship workbench instead defines one adult patient's *first-ever* ICU stay as the candidate index. The supported task is death strictly after a 24-hour prediction point and no later than hospital discharge. It has a narrower allowlist of 19 numeric and three categorical predictor fields. The separation is deliberate: a shared warehouse can serve retrospective description and prospective modeling, but a descriptive field is not automatically eligible at a prospective decision time.

3 Executable study design

3.1 Cohort, prediction time, and outcome

The versioned YAML study specification fixes the study identifier, synthetic origin, first-stay rule, adult threshold, 24-hour horizon, half-open event window, availability policy, task version, feature registry version, random seed, split proportions, and publication cell threshold. Unsupported task or feature-set values are rejected by typed contract validation. Let t_0 denote ICU entry of the patient's first-ever ICU stay and let $t_p = t_0 + 24\,\mathrm{h}$. The first stay is chosen by ICU entry time and then stay identifier *before* age, care-unit, and eligibility filters. A later qualifying stay therefore cannot replace an ineligible first stay.

An eligible row has known t_0, age at admission at least 18 years, and ICU departure strictly after t_p; death at or before t_p is excluded. The optional care-unit restriction is applied after first-stay selection. Positive outcome is death at t_d satisfying $t_p < t_d \leq t_{\text{hospital discharge}}$. The study table contains one eligible ICU stay per selected patient. Cohort preview produces ordered counts for all ICU stays, first stays, adult first stays, care-unit selection, and eligibility at prediction. This flow is a visible accounting mechanism: an apparently strong score cannot silently hide a large loss of eligible patients.

The definition has an important estimand: mortality among adults who survive and remain in intensive care for 24 hours after their first ICU entry. It does not estimate mortality at ICU admission, mortality across all stays, or mortality for patients discharged earlier. The outcome also relies on the local hospital-discharge record. Without a separate follow-up source, an absence of a later death cannot be interpreted as long-term survival.

3.2 Event time and availability time

A prospective predictor requires more than an event occurring before the cutoff. For a chart or laboratory observation with event time t_e and recorded availability time t_a, the implemented rule is

$$t_0 \leq t_e < t_p, \qquad t_e \leq t_a \leq t_p, \qquad t_a \text{ known}. \tag{1}$$

The first interval is half-open, so a measurement made exactly at t_p is excluded. A measurement drawn earlier but stored after t_p is also excluded. Unknown availability is excluded by default. For input events the start time plays the event-time role. The `storetime` field is an operational proxy for availability in this implementation; proving that it matches clinician-visible availability would require source-specific validation on real data. Event filters join chart and input records by the

selected `stay_id`; hospital labs join via admission and are restricted to the selected stay's ICU window. This prevents later events from leaking into the first-stay row.

The feature builder takes means of selected heart rate, blood pressure, respiratory rate, temperature, oxygen saturation, and laboratory measurements, plus 24-hour chart/lab event counts and a first-24-hour vasopressor indicator. Age, gender, admission type, and first care unit supply baseline context. The task registry excludes diagnosis and procedure codes, discharge outcomes, whole-stay summaries, identifiers, future events, and site identifiers from predictors. Diagnosis or procedure information lacking independently established availability is particularly dangerous: hospital discharge coding may describe an outcome after the decision point. The per-feature audit records observed rows, missing rate, distinct values, and an active/inactive variation status. In the current pipeline the model does not automatically drop every inactive feature; the audit makes the issue visible for review.

4 Evaluation, scenarios, and research outputs

4.1 Model and partition protocol

The flagship service uses a fixed patient-level 70/15/15 split with seed 42. Because its cohort has one selected stay per patient, the patient split also separates cohort rows. Numerical inputs receive median imputation and standard scaling; categorical inputs receive most-frequent imputation and one-hot encoding. These preprocessing steps are fitted inside a scikit-learn pipeline on the training partition. The classifier is a class-weighted logistic regression. The validation partition is evaluated, and the test partition is reported separately. Six repeated stratified three-fold scores on the combined development partition are labeled a stability description; they are not an unbiased estimate following an extensive hyperparameter search. The current flagship has one logistic candidate, despite the artifact filename `model_comparison.csv`; that file does not establish a broad model comparison.

Reported discrimination and calibration summaries are AUROC, area under the precision–recall curve (AUPRC), and Brier score. A 200-resample test-row bootstrap estimates 95% percentile intervals when both outcome classes occur. With one selected stay per patient, a row resample is also a patient resample for this task. Undefined one-class cases are marked as not estimable. Coefficients and baseline-centered linear contributions are exported with an additivity check; these are logistic-model explanations on the model-output scale, not causal effects and not output from the external SHAP package.

Illustrative artifact: run `synthetic_icu_mortality_24h_v1-970d76be`	Recorded value
Training / validation / test rows	560 / 120 / 120
Development repeated-CV mean AUPRC (standard deviation)	0.120 (0.027)
Validation AUPRC	0.130
Test AUROC	0.535
Test AUPRC	0.109
Test Brier score	0.241

Table 2: One published aggregate synthetic run. Values are rounded from its `evaluation_summary.json`; its manifest records code revision `e8fdf87`. These scores cannot support a clinical performance claim.

Table 2 is an artifact example, not a pooled estimate or benchmark. Another published local run with the same reported definition hash has different test scores. The present manifest's database snapshot hash is derived from the *list of Silver table names*, not from row contents. Its code revision records the Git commit but not a complete dirty-worktree patch or dependency lock. These facts limit the strength of any claim that two packages represent identical data and executable code. A future publication-grade run should use content-addressed input manifests, a committed or archived

code state, pinned dependencies, and a persisted private split manifest, then explain any intentional stochasticity.

4.2 Controlled data-quality experiments

The workbench includes isolated invented-data scenarios for delayed laboratory availability, duplicate admissions, missing ICU anchors, and impossible oxygen saturation. The intended analysis separates two effects: cohort attrition due to eligibility changes and prediction changes among the same retained patients. A paired comparison command evaluates delayed availability with a frozen baseline model and common held-out patients; scenario gates should block publication when duplicate keys or impossible physiology violate quality rules. These are controlled software challenges. A scenario result is not an estimate of how often an error occurs in a real hospital, and this manuscript makes no numerical claim about paired scenario effects because no current aggregate comparison artifact was inspected for it.

A separate two-site synthetic exercise assigns invented site labels, delays some results at site B, measures available-lab coverage, and fits a separately labeled recalibration on a disjoint site-B calibration partition. It is a transferability demonstration, not an external validation study: both simulated sites arise from one generator and cannot represent independent care systems. A research-grade extension would require genuinely independent, approved sites, explicit mapping of local codes and units, site-specific missingness diagnostics, confidence intervals by site, and a prespecified external evaluation protocol.

4.3 Interfaces and public package boundary

Study validation, preview, run, status, scenario, comparison, and synthetic site analysis are exposed through the command-line workflow; the dashboard provides cohort, timing, quality, and study views. A study-question assistant drafts only supported typed requests from a versioned corpus; it cannot execute arbitrary model code. Published packages use an explicit file allowlist: manifest, cohort and timing flow, feature audit, quality summary, model summary, evaluation summary, linear attributions, and README. SHA-256 checksums support detection of accidental artifact changes. Patient-level predictions and split assignments are not part of the public package. The study contract specifies a minimum cell threshold, but the current publication copier does not apply that threshold to every aggregate field. A release review must check small cells and other disclosure risks before use with restricted data.

5 Validation, limitations, and next research steps

The standard warehouse checks schemas, required fields, key uniqueness, joins, length-of-stay and date sanity, and expected foundational row volume. The research path adds contract, timing, grain, scenario, and publication checks in code and tests. The six-patient seed and 1,000-patient research generator make end-to-end runs repeatable without restricted files. A passing synthetic gate establishes that the implementation behaves as specified on those fixtures. It does not establish that MIMIC-IV columns are complete in all versions, that local timestamps capture real clinical availability, or that the task has clinical utility. Missing optional source files can be represented by empty tables, so operators must check source coverage before interpreting zero events.

The most consequential limits are methodological. The invented generator is based on documented arbitrary rules, not distribution learning from MIMIC-IV or a privacy-preserving real-data synthesis. No real-data distributional similarity or holdout privacy comparison has been performed. The prototype's single ICU mortality task excludes many plausible research questions; its broad workshop tasks include retrospective teaching targets that should not be presented as prospective clinical prediction. The flagship classifier uses simple averages and counts, omitting richer trajectories, code harmonization, unit adjudication, and treatment timing. Observational associations would in any case require careful confounding and missing-data analysis before interpretation. The

API and SQL guardrails have not been evaluated as a production security or disclosure-control system.

There is a concrete staged path for PhD-level work. First, strengthen provenance: hash source content and transformation SQL, persist the split assignment privately, lock dependencies, record the full code state, and verify that two nominally identical studies yield identical cohort and feature fingerprints. Second, validate the time contract against source documentation and clinician workflows, including whether `storetime` truly precedes availability to the intended user. Third, acquire MIMIC-IV through the official credentialing route and execute the predeclared protocol entirely in an approved environment; perform data-quality profiling and inspect exclusion flow before any model comparison. Fourth, introduce a prespecified baseline suite, calibration assessment, subgroup uncertainty, and independent temporal or site validation. Fifth, evaluate synthetic perturbations against measured real-world missingness mechanisms while keeping restricted records and patient-level outputs under the governing data-use agreement. Each step is a proposed study, not an achievement claimed by the current synthetic demonstration.

6 Conclusion

The repository provides a functioning local chain from source-shaped clinical records through warehouse layers to an explicitly timed, aggregate-published research experiment. Its strongest present result is an inspectable *method*: cohort selection, event and availability cutoffs, task-specific predictors, patient separation, quality evidence, and bounded publication can be reviewed together. The observed classifier scores are intentionally modest and scientifically nonclinical because all records in this run were invented. For a prospective supervisor discussion, the platform is best treated as a concrete foundation for research on data provenance, temporal validity, and multi-site transportability, with credentialed MIMIC-IV evaluation and stronger reproducibility controls as the next evidence-producing phase.

Reproducibility note

The study definition is `configs/studies/icu_mortality_24h.yaml`. Its implementation is in `src/research/`, with supporting transforms, SQL, API, and dashboard modules. The invented-data route runs `make research-demo`, then `make study-validate`, `make study-preview`, and `make study-run`. Table 2 identifies its aggregate package by run ID. Reproducing that historical result requires the recorded earlier code state and data inputs. The current `make ci` target runs the local demo, synthetic research flow, flagship study, assistant evaluation, and tests.

References

[1] A. E. W. Johnson et al., "MIMIC-IV, a freely accessible electronic health record dataset," *Scientific Data*, vol. 10, article 1, 2023. doi: 10.1038/s41597-022-01899-x.

[2] A. Johnson et al., "MIMIC-IV," version 3.1, PhysioNet, 2024. doi: 10.13026/kpb9-mt58. Accessed 18 October 2026.

[3] PhysioNet, "MIMIC-IV Clinical Database Demo," version 2.2. https://physionet.org/content/mimic-iv-demo/2.2/. Accessed 18 October 2026.